

הרצת linux embedded בעזרת qemu

1. הורידו גרסת buildroot 2025.02.
2. חלצו את הקבצים וכנסו לתיקיית ה-buildroot
3. הריצו make menuconfig
4. נשנה בתפריט Target options את הארכיטקטורה ל x86_64
5. נסמן בתפריט kernel את linux Kernel וב Defconfig name נרשום x86_64
6. נשמור, נצא מ-buildroot ונריץ make
7. **בינתיים נכין מערכת קבצים מאד קטנה משלנו ולא מ-buildroot:**
הכנת מערכת קבצים:
נפתח חלון טרמינל חדש, ניצור תיקיית minimal_fs ונריץ gedit test.c
בתוכה. נכתוב בקובץ test.c את השורות הבאות:

```
#include <stdio.h>
```

```
void main() {  
    printf("Hello World!\n");  
    while(1);  
}
```

נשמור ונסגור את עורך הקבצים gedit.

נריץ gcc -static test.c -o test

ניצור מערכת קבצים מסוג initramfs (מערכת קבצים ל-RAM) שתכיל קובץ בודד שהוא קובץ ההרצה test שיצרנו:

```
echo test | cpio -o -H newc | gzip > rootfs.cpio.gz
```

cpio זה פורמט לדחיסה שהוא פשוט מ-zip ומ-tar ולכן דורש פחות קוד בקרנל לטובת הפריסה.

בשלב זה אנחנו אמורים לראות בתיקייה minimal_fs את הקבצים: test.c , rootfs.cpio.gz ואת test.

נעתיק את קובץ ה- kernel מתיקיית output/images אשר בתוך תיקיית buildroot לתיקיית minimal_fs (הקובץ bzImage) ואז נריץ בתוך תיקיית minimal_fs:

```
qemu-system-x86_64 -kernel bzImage -initrd rootfs.cpio.gz -append "rdinit=/test"
```

התוצאה שצריך לראות היא חלון של qemu נפתח ולקראת סוף ההדפסות צריך להיות מודפס **Hello World!**.

אם חלון ה- qemu משתלט לכם על העכבר, ctrl + alt ישחרר את זה.

יצירת טרמינל מנוון ע"י fork and exec

צרו קובץ בשם app.c בתיקייה שבה נמצא הקובץ test.c ע"י gedit app.c והעתיקו את הקוד הבא אליו.

```
#include <stdio.h>
```

```
int main(){
```

```
    printf("Operational application is now running\n");
```

```
    return 4;
```

```
}
```

ננסה עכשיו לחקות את הכלי shell (טרמינל)

בקשו מהמרצה שיראה לכם הדגמה של התוצאה הרצויה!

• ערכו את הקובץ test.c כך שיבצע את הפעולות הבאות:

1. ידפיס linux-course-user:/\$

2. יבצע לולאה שבה קוראים בכל איטרציה ל- getc(stdin) עד אשר המשתמש לוחץ על מקש Enter (הערך של Enter ב-hex הוא 0xA).

כל char שקוראים בעזרת getc נכניס לבאפר buf.

3. כשנצא מהלולאה עקב הקשה על enter נקרא ל- fork שתיצור תהליך בן זהה לתהליך ה- test. (לא לשכוח לסגור את המחרוזת ב-buf עם '\0')

4. נבצע בדיקה של ערך החזרה מ- fork ואם אנחנו בקוד של הבן נקרא
execv(buf, NULL); ל-

5. בהמשך הקוד של האב נקרא ל- waitpid על מנת לחכות לסיום ריצת
הבן, נדפיס את ערך ההחזרה של הבן בעזרת המאקרו המתאים שמתואר
ב- man 2 waitpid.

6. נדפיס שוב linux-course-user:/\$

7. נקרא ל- getc(stdin);

8. while(1); על מנת שה- init שלנו יהיה חוקי!

9. על מנת שיהיה קל לדבג את הטיפול בקלט מומלץ להדפיס את buf
ע"י printf("%s", buf);

• נגיע לתיקייה שבה נמצאים שני הקבצים: test.c ו- app.c. נקמפל את שתי
התוכניות בעזרת:

```
gcc --static test.c -o test
```

```
gcc --static app.c -o app
```

ניצור תיקייה חדשה ולתוכה נעביר/נעתיק את שתי התוכניות app ו- test.

מתוך התיקייה החדשה שיצרנו ניצור מערכת קבצים מהם בצורה הבאה:

```
ls | cpio -ov -H newc | gzip > PATH_TO_MINIMAL_FS_DIR/rootfs.cpio.gz
```

נחזור לתיקיית minimal_fs אשר מכילה כבר את ה- linux kernel
(bzImage) ואת שתי האפליקציות שלנו ארוזות במערכת קבצים CPIO ואז
נריץ:

```
qemu-system-x86_64 -kernel bzImage -initrd rootfs.cpio.gz -append "rdinit=/test quiet"
```